UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA

RELATÓRIO DO TRABALHO EM GRUPO

DE

SISTEMA DE GESTÃO DE BASE DE DADOS II

CLASSE: 3º ANO

Integrantes do Grupo:

António Almeida K. Nzinga

Kiassisua Simão Pedro

Luís Mucuzo António

Maria Tumba Domingos Jão

Suzana Diasivi Nicolao

DOCENTE

_____

Moyo Kanivengidio

UÍGE 2025/2026

Índice

Introdução 3

Entidades Principais 4

Evento 4

Relações e Cardinalidades 4

Restrições de Integridade 4

Esquema Relacional 4

Implementação da Base de Dados 5

Exemplo de Script DDL (MySQL) 5

Consultas e Relatórios 6

Eventos com oradores e número de participantes 6

Participantes em mais de um evento **6**

Sessões de um evento específico **7**

Receita total por event**o** 7

Oradores em eventos de diferentes cidades **7**

Alguns Desafios 7

Conclusão 8

Referência 9

Artigos Acadêmicos 9

Introdução

A organização de eventos académicos, como conferências e workshops, exige um sistema eficiente para gerir participantes, oradores, sessões e inscrições. Este relatório apresenta o processo de análise, modelagem, implementação e testes de um sistema de gestão de eventos académicos, utilizando bases de dados relacionais (MySQL ou SQL Server). O objetivo é garantir robustez, integridade e escalabilidade, permitindo consultas e relatórios úteis para a tomada de decisão.

Entidades Principais

Evento: Nome, DataInicio, DataFim, Local, Capacidade, PreçoInscricao

- Participante: Nome, Email, Instituição, Telefone
- Orador: Nome, Email, Instituição, Especialidade
- Sessão: Título, DataHora, Duração, Sala
- Inscrição: DataInscricao, StatusPagamento

Relações e Cardinalidades

- Um Evento possui muitas Sessões.
- Um Evento possui muitos Participantes via Inscrições.
- Um Evento possui muitos Oradores.
- Um Participante pode inscrever-se em vários Eventos.
- Um Orador pode participar em vários Eventos.

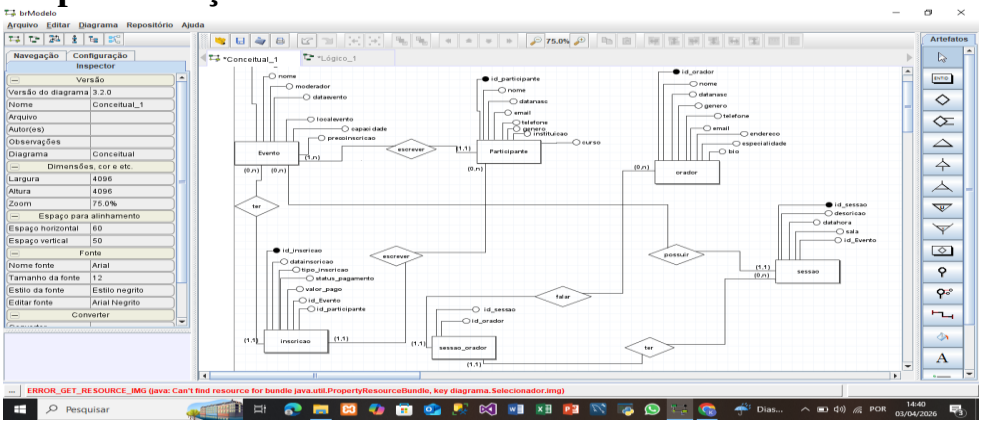
Restrições de Integridade

- `DataFim > DataInicio` em Evento.
- `Capacidade > 0`.
- `PreçoInscricao >= 0`.
- Emails únicos para Participantes e Oradores.

Esquema Relacional

- Evento(EventoID PK, Nome, DataInicio, DataFim, Local, Capacidade, PreçoInscricao)
- Participante(ParticipanteID PK, Nome, Email UNIQUE, Instituição, Telefone)
- Orador(OradorID PK, Nome, Email UNIQUE, Instituição, Especialidade)
- Sessão(SessaoID PK, EventoID FK, Título, DataHora, Duração, Sala)
- Inscrição(InscricaoID PK, EventoID FK, ParticipanteID FK, DataInscricao, StatusPagamento)
- EventoOrador(EventoID FK, OradorID FK, PRIMARY KEY(EventoID, OradorID))

Implementação da Base de Dados



Script DDL (MySQL)

```
create table Evento (
id_Evento int primary key auto_increment,
nome varchar (100) not null,
moderador varchar (100) not null,
data_evento datetime CHECK (datainicio < datafim),
localevento varchar (100) not null DEFAULT 'Angola' CHECK
(LENGTH(localevento) >= 5),
capacidade int,
precoinscricao decimal (10,2) DEFAULT 0.00 CHECK (precoinscricao >= 0));

create table Participante (
id_participante int primary key auto_increment,
nome varchar (100) not null,
datanasc date,
email varchar (100) not null UNIQUE,
telefone varchar(20) default '+244 ' unique CHECK (telefone LIKE '+244 %' AND
LENGTH(telefone) = 14),
genero enum ('M', 'F'),
instituicao varchar (100),
curso varchar (100));

create table orador (
id_orador int primary key auto_increment,
nome varchar (100) not null,
datanasc date,
genero enum ('M', 'F'),
telefone varchar(20) default '+244 ' unique CHECK (telefone LIKE '+244 %' AND
LENGTH(telefone) = 14),
email varchar (100) not null UNIQUE,
endereco varchar (100),
especialidade varchar (100),
bio varchar (5000) not null);
```

```
create table sessao (
id_sessao int primary key auto_increment,
descricao varchar (100) not null,
datahora datetime,
sala varchar (100),
id_Evento int,
foreign key(id_Evento) references Evento (id_Evento));

create table inscricao (
id_inscricao int primary key auto_increment,
datainscricao date,
comprovativo varchar(100),
tipo_inscricao enum('Gratuita', 'Paga') not null,
status_pagamento  enum('Pendente', 'Confirmado', 'Cancelado', 'Isento') default
'Pendente',
valor_pago decimal (10,2) default 0.00 check (valor_pago >= 0),
id_Evento int,
id_participante int,
Foreign key (id_evento) references evento(id_Evento),
Foreign key (id_participante) references Participante(id_participante));

Create table sessao_orador (
   id_sessao int,
   id_orador int,
primary key (id_sessao, id_orador),
   foreign key(id_sessao) references sessao(id_sessao) on delete cascade,
   Foreign key (id_orador) references orador(id_orador)on delete cascade

);

##depois de executar as tabela , executei o gatilho a baixo em seguida
```

Consultas e Relatórios

Eventos com oradores e número de participantes

sql

```
SELECT e.Nome, COUNT(DISTINCT i.ParticipanteID) AS
TotalParticipantes,
       GROUP_CONCAT(DISTINCT o.Nome) AS Oradores
FROM Evento e
LEFT JOIN Inscricao i ON e.EventoID = i.EventoID
LEFT JOIN EventoOrador eo ON e.EventoID = eo.EventoID
LEFT JOIN Orador o ON eo.OradorID = o.OradorID
GROUP BY e.EventoID;
```

Participantes em mais de um evento

sql

```
SELECT p.Nome, COUNT(DISTINCT i.EventoID) AS NumEventos
FROM Participante p
JOIN Inscricao i ON p.ParticipanteID = i.ParticipanteID
GROUP BY p.ParticipanteID.
```

```
HAVING COUNT(DISTINCT i.EventoID) > 1
```

Sessões de um evento específico

sql
```
SELECT s.Titulo, s.DataHora, s.Sala
FROM Sessao s
JOIN Evento e ON s.EventoID = e.EventoID
WHERE e.Nome = 'Conferência IA 2026'
ORDER BY s.DataHora;
```

Receita total por evento

sql
```
SELECT e.Nome, COUNT(i.InscricaoID) * e.PrecoInscricao AS
ReceitaTotal
FROM Evento e
JOIN Inscricao i ON e.EventoID = i.EventoID
GROUP BY e.EventoID;
```

Oradores em eventos de diferentes cidades

sql
```
SELECT o.Nome, COUNT(DISTINCT e.Local) AS NumCidades
FROM Orador o
JOIN EventoOrador eo ON o.OradorID = eo.OradorID
JOIN Evento e ON eo.EventoID = e.EventoID
GROUP BY o.OradorID
HAVING COUNT(DISTINCT e.Local) > 1;
```

Alguns Desafios

Durante o processo de design e implementação, os principais desafios foram:

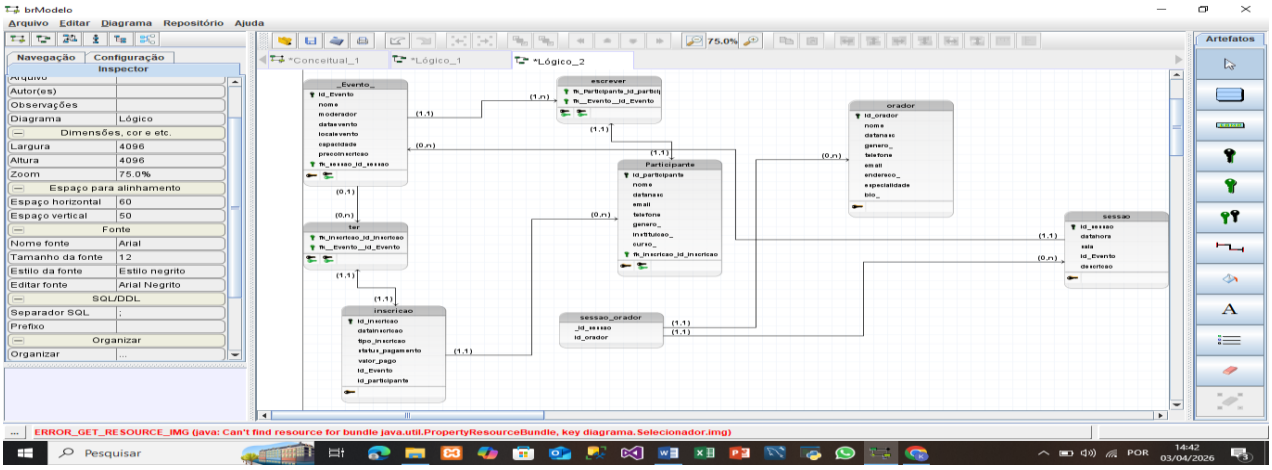
- Garantir integridade referencial entre múltiplas entidades relacionadas.
- Normalização sem perda de informação, evitando redundâncias.
- Criação de dados realistas para testes, assegurando diversidade de casos.

Esses desafios foram superados com uma análise cuidadosa das dependências funcionais, aplicação rigorosa das formas normais e uso de restrições SQL (CHECK, UNIQUE, FOREIGN KEY). O resultado é um sistema robusto, escalável e capaz de responder a consultas complexas de forma eficiente.

Conclusão

O desenvolvimento do sistema de gestão de eventos académicos permitiu aplicar de forma prática os conceitos fundamentais de modelagem e implementação de bases de dados relacionais. A análise de requisitos identificou claramente as entidades centrais (Evento, Participante, Orador, Sessão e Inscrição) e suas inter-relações, garantindo um desenho consistente e fiel à realidade.

Referência



Artigos Acadêmicos

- OLIVEIRA, J. P.; SANTOS, R. M. *Modelagem de banco de dados para sistemas de gestão de eventos acadêmicos*. Revista de Sistemas de Informação, v. 12, n. 2, p. 45-60, 2020.
- PEREIRA, L. A.; COSTA, M. F. *Uso de SQL Server na implementação de sistemas de conferências acadêmicas*. Anais do Congresso Nacional de Informática, 2019.